

Guía de cierre y rúbrica del examen suspenso

Equipo ACC — TicketFold, soporte técnico distribuido para un proveedor de Internet

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/PFC-SOPORTE-ISP
Cabecera evaluada	Rama main, commit 964fa6e — 198 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Pacheco Cárdenas Cristhian Daniel · Carpio Mendoza Carlos José · Álvarez Párraga Jeremy Alexis · Cando Moreno Robinson Rodrigo
Rinden el examen suspenso	Álvarez Párraga Jeremy Alexis
Compleitud del proyecto	55 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: al 16/09 el repositorio figura bajo [gleiston-guerrero/PFC-SOPORTE-ISP](https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP); cumple si sigue así al corte. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	75 %	Véase §1
2	Estructura del repositorio y archivos de raíz	Por modificar	Por modificar	70 %	Véase §2
3	Línea base congelada e integración	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	65 %	Véase §4
5	Patrones GoF y ADR	Por modificar	Por modificar	70 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	75 %	Véase §6
7	Calidad de la aplicación web	Por modificar	Por modificar	75 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	65 %	Véase §8
9	Calidad de la aplicación móvil	Por modificar	Por modificar	55 %	Véase §9
10	Integración y contratos	Por culminar	Por culminar	45 %	Véase §10
11	Persistencia distribuida	Por modificar	Por modificar	70 %	Véase §11
12	Pirámide de pruebas	Por culminar	Por culminar	40 %	Véase §12

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
13	Pipeline CI/CD	Hecho	Por modificar	85 %	Véase §13
14	Observabilidad	Por modificar	Por modificar	65 %	Véase §14
15	Protocolo y campaña experimental	Por culminar	Por culminar	35 %	Véase §15
16	Paquete de datos y reproducibilidad	Por modificar	Por culminar	35 %	Véase §16
17	Evaluación ISO/IEC 25010	Por culminar	Por culminar	30 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por culminar	35 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	60 %	Véase §19
20	Trabajo en equipo y autoría	Por culminar	Por culminar	25 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

El pipeline es el más completo del curso: los siete jobs en DAG más `compile-latex` y `verify-checksums`; `build-images` depende de `lint`, las tres suites y la integración, y publica siete imágenes en GHCR etiquetadas por commit; la integración levanta el stack con sonda de salud y verifica el proveedor Pact y Playwright contra servicios reales; la ejecución sobre `964fa6e` terminó con 15 jobs en verde. La SPA, recompilada desde cero, pasa `tsc` y ESLint sin advertencias y 75 pruebas con cobertura de líneas 87,6 % (1.583/1.807) y de ramas 89,09 % (237/266) sobre todo `src/`; tiene las cinco rutas protegidas, `i18n` es/en, tema claro y oscuro y JWT en `sessionStorage` con expiración. El móvil guarda el JWT cifrado, tiene caché Room y actualización por arrastre y captura foto y GPS reales; el APK está firmado desde un secreto en CI. Command, Chain of Responsibility, Observer de escalado, Factory Method y Strategy de SLA y correlación son código real con 86 pruebas. Las cuatro métricas nuevas y tres `crdb_*`, registros JSON con `trace_id` y trazas hacia Tempo están instrumentados. Los tres manifiestos verifican (3/3, 41/41, 1/1) en CI; `generar_reporte_correl.py` regenera el análisis y la tabla del documento idénticos; `.gitattributes` deja cero archivos CRLF. El manuscrito compila a 55 páginas sin indefinidas, su PDF corresponde a la fuente y la tabla de aporte ya incluye a los cuatro integrantes.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando

a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica en CITATION.cff, portada y diapositiva; README con equipo, roles y avisos vigentes	1	1	2	1	5	2,3	Por modificar	0,92
§2	Raíz sin subproductos de compilación ni carpetas heredadas, mapa del Listado 3, rutas citadas existentes y variables del móvil	2	2	3	2	9	4,1	Por modificar	1,64
§3	Integración por PR con revisión de otro integrante, rama principal sin re-escrituras, CHANGELOG.md y etiqueta anotada de cierre	2	2	2	2	8	3,7	Por culminar	0,74
§4	Dominio sin Spring ni setters públicos, cuatro capas en todos los servicios con lógica, controladores sobre abstracciones y ArchUnit	3	3	4	3	13	5,9	Por modificar	2,36
§5	Strategy de rutas y Observer de SLA con uso real y pruebas; ADR-005 corregido y ADR-006 con comparación cuantitativa	3	3	3	3	12	5,5	Por culminar	1,10
§6	Vista de SLA por zona y por técnico sobre un endpoint real y rol de coordinador	2	3	3	3	11	5,0	Por culminar	1,00
§7	Compuerta de cobertura web ≥ 70 % en CI y artefacto dist versionado por commit	1	2	2	1	6	2,7	Por modificar	1,08
§8	Cierre en sitio que envía y persiste foto y GPS (con cola sin conexión) y listado de tickets asignados al técnico	3	4	4	4	15	6,8	Por culminar	1,36
§9	Pruebas de los tres ViewModels, E2E instrumentada login→lista→detalle y APK en Release inmutable de la etiqueta	2	3	3	3	11	5,0	Por culminar	1,00
§10	Contrato del consumidor móvil, contrato web ampliado a todas las operaciones y OpenAPI con rutas absolutas, esquemas y validación en CI	3	3	3	3	12	5,5	Por culminar	1,10
§11	Migraciones versionadas con Flyway, imágenes con versión fija y experimento de caída reproducible en la configuración E4	2	3	3	3	11	5,0	Por modificar	2,00
§12	Cobertura backend ≥ 70 % con compuerta en los cuatro servicios Java, Testcontainers en auth y report e informes como artefactos	3	3	4	4	14	6,4	Por culminar	1,28
§13	APK dependiente de lint y pruebas y agente OpenTelemetry con versión fija	1	2	2	1	6	2,7	Por modificar	1,08

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§14	Registros JSON y traceparent en el móvil, latencia medida en el cliente, histogramas y evidencia del tablero bajo carga	3	3	3	3	12	5,5	Por modificar	2,20
§15a	Protocolo versionado antes de medir y campaña CORREL con $r = 10$, variabilidad real, unidad de análisis correcta e IC exactos	4	3	3	4	14	6,4	Por culminar	1,28
§15b	Carga Locust conforme: 50 usuarios $\times$ 5 min y rampa 0→200 en 10 min, diez corridas por escenario	2	3	2	3	10	4,6	Por culminar	0,92
§16	Un juego canónico de resultados, manifiestos que cubren todos los datos, diccionario de datos y reproducción verificada en CI	3	2	3	2	10	4,6	Por culminar	0,92
§17	ISO/IEC 25010 completa: fiabilidad de 1 h, seguridad sobre todos los endpoints, mantenibilidad y compatibilidad con artefactos crudos	4	3	3	4	14	6,4	Por culminar	1,28
§18	Manuscrito con cifras canónicas, figuras de observabilidad y C4 nivel 3, estilo que imprima DOI/ISBN y versión congelada	2	2	4	3	11	5,0	Por culminar	1,00
§19	Amenazas de las campañas E4 con mitigación aplicada y reflexión ética $\geq \frac{1}{2}$ página con principios ACM	3	1	2	2	8	3,7	Por modificar	1,48
§20	Autoría transparente (.mailmap, declaración de commits por guion y reatribuidos), actas contemporáneas y aporte propio verificable	2	1	2	2	7	3,2	Por culminar	0,64
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **2,64 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00.

3. Trabajo exigido

§1 Identificación del entregable y URL

Qué encontré: CITATION.cff (repository-code) y docs/diapositivas/identificacion_lms.tex siguen apuntando a carlospatroner-boop/PFC-SOPORTE-ISP; el manuscrito no contiene ninguna URL del repositorio; el README no tiene la URL, ni los integrantes, ni sus roles (sólo están en PLAN.md, un plan de la E3), conserva el aviso de la “fecha límite del viernes 28 de agosto” y dice que el documento tiene 51 páginas cuando tiene 55; CITATION.cff no declara versión. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP> en README, CITATION.cff, portada del manuscrito y diapositiva; sección “Equipo” en el README con nombre completo, rol y usuario de GitHub; version y date-released coherentes con la etiqueta de cierre. **Cómo lo verifico:** `git grep -n carlospatroner-boop v4.0.0` no devuelve nada; `git grep -n gleiston-guerrero/PFC-SOPORTE-ISP v4.0.0 -- README.md CITATION.cff docs/latex/main.tex` devuelve las tres.

§2 Estructura — subproductos, carpetas heredadas y rutas que no existen

Qué encontré: versionan `main.bbl` y `main.blg`, 194 archivos HTML generados por JaCoCo y un `.exec` de 1,33 MB; `node_modules` de `notification-service` (1.575 archivos) sigue en el historial; conviven carpetas heredadas o ajenas sin explicación (`frontend/` de la E3, `PLAN.md`, `start-all.ps1`, `Evidencias de Modificaciones/` con capturas de “clic en editar”). Faltan ADR-001 y ADR-002, el C4 de nivel 3 en imagen, `iso25010.csv` y `boxplot_latencia.png` del Listado 3, y los siete servicios tienen nombres distintos sin tabla de correspondencia. `tolerancia_fallos.md` remite a `docs/evidencias/frames/` y una prueba E2E cita `rebalance_demo_tickets.sql`: ninguno existe. `.env.example` no tiene variables del móvil, cuya URL base `10.0.2.2:8000` está fija en el código. **Qué deben hacer:** retirar subproductos y añadir `*.bbl`, `*.blg` y `*.exec` a `.gitignore`; conservar de JaCoCo sólo `jacoco.xml` y `jacoco.csv`; mover lo heredado a `docs/entrega3/` o retirarlo; añadir al README la tabla “Listado 3 → ruta real”; crear los ADR ausentes o justificar su ausencia; corregir las rutas rotas; declarar la URL base del móvil como `buildConfigField` documentada en `.env.example`. **Cómo lo verifico:**

```
git ls-files | grep -E "\.(bbl|blg|aux|log|exec)$|jacoco-resources|node_modules" # vacío
grep -rn "frames/|rebalance_demo_tickets" --include=*.md --include=*.ts . # vacío
```

§3 Línea base — sin etiqueta de cierre, sin revisión y con historia reescrita

Qué encontré: las cuatro etiquetas existentes son ligeras y ninguna apunta a un commit de `main` (tres son copias de seguridad de reescrituras y `mobile-release` apunta a `f5300ed`, fuera de la rama). No hay `CHANGELOG`. De 13 PR anteriores al corte sólo el #1 tiene revisiones; cinco los abrió y fusionó la misma persona. Los 45 commits posteriores al último PR entraron directamente en `main`, y la rama se reescribió con *force-push* al menos tres veces (16 commits descartados el 09/09, siete reaplicados el 10/09 con otros autores y uno eliminado el 13/09). Los archivos del Release `mobile-release` se reemplazaron después del corte, así que no prueban nada sobre la entrega. **Cómo debe quedar:** rama `main` protegida (PR obligatorio, una aprobación, sin *force-push*); cada cambio del suspenso en un PR aprobado por un integrante distinto del autor; `CHANGELOG.md` con entradas fechadas de v1.0.0 a v4.0.0 y una del examen suspenso; etiquetas `backup-*` retiradas; etiqueta anotada de cierre sobre el último commit de `main`, creada **al final de todo**, con un Release inmutable ligado a ella. **Cómo lo verifico:**

```
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main
git for-each-ref refs/tags --format='%(%objecttype) %(%refname:short) %(%taggerdate:iso)'
git ls-remote --tags origin v4.0.0 # deben salir DOS líneas
```

y en la API, `/pulls?state=closed` con una revisión `APPROVED` de otro usuario en cada PR.

§4 Backend en capas — Spring dentro del dominio

Qué encontré: nueve archivos de `svc-principal/.../domain` llevan anotaciones de Spring (`@Component("c0")`, `@Component("c1")`, `@Value`, `@Order(1)`, en estrategias, cadena de escalado, fábrica y políticas de SLA), igual que `TelemetryStore`, lo que contradice el propio ADR-005 (“el dominio queda libre de anotaciones JPA/Spring”). Las entidades `Ticket`, `Incidencia`, `User`, `RefreshToken` y `TicketSummary` son anémicas con `@Setter` y `@AllArgsConstructor`, así que cualquier invariante se puede saltar. `telemetry-service` no tiene `application` ni `presentation`; `notification-service` y `ai-service` son planos. `TicketController` depende de tres manejadores concretos y no de `TicketCommandHandler<C,R>`. **Cómo debe quedar:** dominio sin `org.springframework`, `jakarta.persistence` ni setters públicos; beans contruidos en `infrastructure/config`; entidades con métodos de negocio (`asignar()`, `cambiarEstado()`, `escalar()`); cuatro capas en cada servicio con lógica, o un ADR que justifique los triviales; controladores sobre interfaces; prueba `ArchUnit` que lo haga cumplir. **Cómo lo verifico:**

```
grep -rln "import org.springframework\\lombok.Setter" services/*/src/main/java/ec/edu/uteq/soporte/*/domain #
↪ vacío
mvn -B test -Dtest=ArchitectureTest # en svc-principal, auth-service, report-service y telemetry-service
```

§5 Patrones GoF — faltan los dos que dependen del dominio del ISP

Qué encontré: Repository, Factory Method, Command, Chain of Responsibility, Observer de escalado y Strategy de SLA y de correlación son código real con pruebas. Pero la Tabla 1 exige un **Strategy de rutas** y no existe ninguna estrategia de rutas ni de orden de visitas en el backend ni en el móvil, y el **Observer de SLA** sólo cubre el escalado: ningún sujeto notifica a la consola ni al móvil el paso de un ticket a “en riesgo” o “vencido”. ADR-005 repite casi literalmente un párrafo del contexto, afirma que el dominio está libre de Spring (falso, §4) y cita 45 pruebas cuando hay 86. ADR-006 aporta cifras sólo de la opción elegida (APK de 11,06 MB y 20 dependencias) y reconoce que no midió ninguna alternativa. **Cómo debe quedar:** RutaStrategy con al menos dos implementaciones (p. ej. cercanía por GPS y prioridad por SLA) usadas por un caso de uso que expone la ruta del técnico y que el móvil consume, con pruebas; Observer de SLA que publique el cambio de estado de SLA hacia métrica, mensajería y consola, con pruebas; ADR-005 corregido y ADR-006 con una tabla de al menos dos criterios medidos o citados *para ambas opciones*. **Cómo lo verifico:** `grep -rn "RutaStrategy" services/svc-principal/src/main | wc -l ≥ 3`; `mvn -B test -Dtest='*Ruta*Test,*Sla*Observer*Test'` en verde; `grep -c "Antes de este refactor" docs/adr/0005-patrones-gof.md` da 1.

§6 Aplicación web — la vista de SLA es sólo un subtítulo

Qué encontré: la Tabla 1 pide una consola de operadores y coordinadores con vista de SLA por zona y por técnico. `es.json` tiene el subtítulo “Vista de SLA por zona y por técnico”, pero `ConsolePage.tsx` muestra tres indicadores globales, un filtro de zona y una insignia por fila; `ReportsPage.tsx` desglosa por estado, zona y categoría, sin técnico ni porcentaje de cumplimiento; `SummaryResponse` no expone agrupación por técnico. No existe el rol de coordinador u operador (roles: CLIENTE, TECNICO, ADMIN). **Cómo debe quedar:** una vista con tickets abiertos, vencidos, porcentaje de cumplimiento de SLA y tiempo medio de resolución por zona y por técnico, alimentada por un endpoint real del modelo de lectura (p. ej. `GET /api/v1/reports/sla?groupBy=zone|technician`), con i18n, estados de carga y error, pruebas Vitest y una prueba Playwright; rol de coordinador con acceso a ella. **Cómo lo verifico:** `grep -rn "groupBy\|byTechnician" services/report-service/src/main apps/web/src` NO VACÍO; `npx vitest run src/features/sla`; la respuesta del endpoint trae el porcentaje por técnico.

§7 Calidad web — la cobertura no es una compuerta

Qué encontré: la cobertura real es buena (véase el recuadro de lo hecho), pero el CI ejecuta `npm test` sin `--coverage` y `vitest.config.ts` no define umbrales: una caída por debajo del 70 % no rompería el pipeline. Ningún job publica `dist/` versionado por commit. **Qué deben hacer:** `coverage.thresholds` de 70 en las cuatro métricas; `npm run test:coverage` en el job con el informe como artefacto; paso de `npm run build` que suba `web-dist-{{ github.sha }}` con su suma; demostrar en un PR que la compuerta falla al subir temporalmente el umbral. **Cómo lo verifico:** `grep -n thresholds apps/web/vitest.config.ts`; artefactos `web-dist-<sha>` y `web-coverage` en la ejecución del commit final.

§8 Aplicación móvil — la evidencia se captura y se descarta

Qué encontré: inicio de sesión, listado con actualización por arrastre, detalle, caché Room y JWT cifrado funcionan, y la foto y el GPS se capturan de verdad. Pero `TicketDetailViewModel.closeOnSite()` sólo comprueba que existan y luego llama a `ticketRepository.closeOnSite(ticketId)` **sin foto ni coordenadas**; el propio `TicketRepository.kt` lo reconoce en un comentario (“hoy solo actualiza el estado a RESUELTO”), no hay endpoint multipart ni de evidencia en la app ni en el backend, y aun así el diálogo le dice al técnico que “se enviará la evidencia fotográfica y las coordenadas GPS”. Además el listado no muestra los tickets *asignados*: el backend filtra al técnico por zona, así que ve todos los de su zona. No hay

registros estructurados (Timber). **Cómo debe quedar:** cierre en sitio que sube la foto y la latitud y longitud (multipart u objeto almacenado), persiste la evidencia en una tabla creada por migración (§11) con ticket, técnico y marca de tiempo, la muestra en el detalle web y, sin conexión, la encola y sincroniza; listado móvil restringido a los tickets asignados al técnico autenticado. **Cómo lo verifico:**

```
grep -n "Multipart\\|latitude" apps/mobile/app/src/main/java/**/data/remote/TicketApi.kt
grep -rn "close-on-site\\|ticket_evidence" services/svc-principal/src
./gradlew :app:testDebugUnitTest --tests '*TicketDetailViewModelTest*'
```

más la evidencia versionada de un cierre real (petición y respuesta del gateway y fila en la tabla de evidencias).

§9 Calidad móvil — tres pruebas unitarias y ninguna E2E

Qué encontré: sólo LoginViewModelTest (3 pruebas); TicketListViewModel y TicketDetailViewModel, que contiene la lógica de cierre, no tienen pruebas. La única instrumentada valida campos vacíos del formulario y su comentario aclara que no depende del backend. El APK versionado se subió a mano; el Release depende de una etiqueta ligera fuera de main y sus archivos se sobrescriben en cada *push*. **Cómo debe quedar:** pruebas de los tres ViewModels (éxito, error, sin conexión y cierre con evidencia) con JUnit 5, coroutines-test y mockk; al menos una E2E instrumentada con servidor simulado que recorra inicio de sesión, lista y detalle; Release publicado sólo al empujar la etiqueta de cierre, sin sobrescritura, con SHA256SUMS.txt y enlace desde el README. **Cómo lo verifico:** `grep -c "@Test" apps/mobile/app/src/test/**/*ViewModelTest.kt` (tres archivos, 10 o más pruebas); `grep -rn MockWebServer apps/mobile/app/src/androidTest`; el APK del Release verifica con `sha256sum -c y apksigner verify`.

§10 Contratos — el cliente móvil no tiene contrato

Qué encontré: un único consumidor Pact, la web, con dos interacciones (ambas `GET /api/v1/tickets`: 401 sin testigo y 200 con ADMIN); no cubre detalle, creación, cambio de estado, asignación, inicio de sesión ni reportes. El móvil usa tres operaciones sin ningún contrato. La verificación del proveedor contra el stack real es correcta, pero sólo sobre esas dos interacciones. `docs/api/openapi.yaml` usa rutas relativas ambiguas (`/`, `/id`, `/summary`) bajo diez `servers` globales, deja sin esquema las respuestas 2xx de listar, cambiar estado, asignar y administrar usuarios, no documenta `GET /api/v1/tickets/incidencias`, y nada lo valida en CI. **Cómo debe quedar:** dos consumidores (soporte-web y soporte-mobile) con una interacción por cada operación que usa cada cliente, incluida la del cierre en sitio de §8, verificados por el proveedor real en CI; OpenAPI con rutas absolutas, esquemas en todas las respuestas y `npx @redocly/cli lint` en el job de lint. **Cómo lo verifico:** `ls pacts/` con dos archivos (web ≥ 5 y móvil ≥ 4 interacciones); registro del job de integración con “Verifying a pact between soporte-mobile and ticket-service” en verde; `redocly lint` sin errores.

§11 Persistencia distribuida — esquema sin historial de versiones

Qué encontré: tres nodos CockroachDB con localidades, healthchecks y `num_replicas = 3`. Pero no hay migraciones versionadas: el esquema se aplica con scripts `CREATE TABLE IF NOT EXISTS` desde el `command` de `db-init`, hay un `ALTER TABLE auth_db.users ADD COLUMN ... zone` incrustado en `docker-compose.yml`, y `docs/db/schema.sql` es una copia “de referencia” que puede divergir. La imagen es flotante (`latest-v23.2`), también en Testcontainers. La evidencia de tolerancia a fallos es de la E3 y cita capturas inexistentes. **Cómo debe quedar:** migraciones Flyway (`V1__init.sql`, `V2__users_zone.sql`, ...) en cada servicio con base de datos y `db-init` reducido a inicializar el clúster; versión fija de CockroachDB en compose y pruebas; script que detiene un nodo bajo carga y guarda un CSV con marcas de tiempo y su manifiesto, ejecutado sobre la configuración E4. **Cómo lo verifico:** `git ls-files "services/*/src/main/resources/db/migration/V*.sql"`

(≥ 2 por servicio con base); `grep -n "ALTER TABLE\\|latest" docker-compose.yml Vacío; sha256sum -c db-cluster/results/SHA256SUMS.txt.`

§12 Pirámide de pruebas — la cobertura del backend no se mide en CI

Qué encontré: 121 pruebas Java (86 en `svc-principal`, 15 `auth`, 11 `report`, 8 `telemetry`, 1 `gateway`), 75 `web`, 3 E2E Playwright, 3 unitarias y 2 instrumentadas móviles. Sólo `svc-principal` mide cobertura, en un perfil que el CI no activa y sin `check`: el 81,1 % de líneas procede de una ejecución local del 07/09 sobre un commit que no está en `main`, y sus ramas están en 57,1 %; `auth`, `report` y `telemetry` no tienen JaCoCo. Testcontainers sólo en tres pruebas de `svc-principal`. La carga no sigue la guía (§15b). El CI sólo sube el informe de Playwright. **Cómo debe quedar:** JaCoCo con `prepare-agent`, `report` y `check` al 70 % (instrucciones, ramas y líneas) ligados a `test` en los cuatro servicios Java, ejecutado en CI; pruebas de repositorio con Testcontainers en `auth` y `report`; informes de surefire, JaCoCo, Vitest, pruebas móviles y Playwright como artefactos. **Cómo lo verifico:**

```
grep -rn "<minimum>0.70" services/*/pom.xml | wc -l          # >= 4
for s in auth-service svc-principal report-service telemetry-service; do (cd services/$s && mvn -B test) ||
↪ echo FALLA $s; done
```

§13 Pipeline CI/CD — dos ajustes

Qué encontré: el pipeline es correcto (véase lo hecho) y baja sólo por lo que protege: no tiene compuertas de cobertura (§7, §12) ni verifica el contrato móvil (§10). Además `build-mobile-apk` no depende de lint y los Dockerfile descargan el agente OpenTelemetry desde `releases/latest`, lo que hace la imagen no reproducible. **Qué deben hacer:** `needs: [lint, test-mobile]` en el job del APK; URL del agente con versión fija y verificación de su suma. **Cómo lo verifico:** `grep -n "releases/latest" services/*/Dockerfile Vacío.`

§14 Observabilidad — el cliente móvil no emite nada

Qué encontré: el panel “latencia extremo a extremo desde el cliente móvil” consulta la latencia del gateway en el servidor; el móvil no emite métricas, trazas ni registros JSON con `trace_id`. `notification-service` y `ai-service` no exportan registros al Collector, que no tiene pipeline de registros. Los paneles de latencia promedian cuantiles de `summaries` entre instancias, lo cual no es válido estadísticamente. No hay ninguna captura del tablero ni de una traza, ni exportación de Prometheus, bajo carga. **Cómo debe quedar:** interceptor `OkHttp` que genere `traceparent` y mida la duración de cada petición, exportada por OTLP; Timber con árbol JSON que incluya `trace_id`; `http_request_duration_seconds` como histograma consultado con `histogram_quantile`; carpeta `docs/evidencias/observabilidad/` con capturas de los seis paneles y de una traza del móvil a la base de datos y respuestas `query_range` en JSON, todas con marcas de tiempo dentro de la campaña Locust de §15b.

§15 Campañas experimentales — diseño no cumplido

Qué encontré (CORREL): 60 filas = 3 escenarios $\times$ 3 modos $\times$ 5 réplicas, no 10; las cinco réplicas de cada una de las 12 celdas son **idénticas valor a valor** ($DE = 0$) y `c1` y `c2` coinciden fila a fila, porque el sistema es determinista y repetir no añade variabilidad; la población es de 20/60/20+20 abonados frente a los 100/500/100+100 del protocolo, y falta el escenario “Esc-4 en la misma zona” del ADR-0008. `protocolo-e4.md` entró **en el mismo commit que los datos** (8869188), con los resultados ya escritos, y el ADR-0008 no tiene hipótesis ni umbrales; aun así el documento dice “se rechaza H_0 ”. El intervalo “IC95 binomial [0,0]” es un Wald degenerado (con 0/30, Clopper-Pearson da [0; 0,116]); las “30 corridas” de Esc-4 son 15 corridas por dos zonas; el Mann-Whitney con $n = 20$ trata filas no independientes como réplicas; y la única marca de tiempo de la campaña

se borró del JSON el 10/09. **Qué encontré (Locust):** cinco corridas por escenario, con máximo de 30 usuarios durante 58–60 s (se exige 50 durante 5 minutos) y rampa de 0 a 60 en 119 s (se exige 0 a 200 en 10 minutos); tras descartes quedan $n = 3$; la columna de error etiquetada como 5xx incluye `ConnectionResetError`. **Cómo debe quedar:** protocolo con hipótesis, variables, umbrales, α , poblaciones del ADR y plan de análisis versionado *en un commit anterior al primer dato*; CORREL con $r = 10$ por celda, semilla distinta por réplica, orden aleatorio de modos, marcas de inicio y fin por réplica y árbol limpio registrado; unidad de análisis = corrida; intervalos exactos; Locust con `-u 50 -t 5m` y rampa 0→200 en 600 s, diez corridas por escenario. El cambio de prueba estadística de la E3 (hecho el 08/09) se declara como desviación del protocolo. **Cómo lo verifico:**

```
git log --format='%h %ad' --date=iso -- docs/experimentos/protocolo-e4.md | tail -1 # anterior a:
git log --format='%h %ad' --date=iso -- experimentos/resultados/correlacion_corridas.csv | tail -1
python -c "import csv,collections as c;r=list(csv.DictReader(open('experimentos/resultados/correlacion_corrid
↪ as.csv')));print(c.Counter((x['escenario'],x['modo']) for x in r))"
```

y en cada `*_stats_history.csv`: máximo de `User Count` 50 con duración ≥ 300 s, o 200 con ≥ 600 s.

§16 Paquete de datos — manifiestos parciales que se regeneran solos

Qué encontré: los manifiestos verifican, pero dejan fuera `spark/results/**`, `db-cluster/results/*`, `docs/experimentos/resultados/*`, los datos de JaCoCo, la tabla generada del documento y los guiones de `experimentos/`; y `correr_campana.py` y `analizar_resultados.py` regeneran el manifiesto al final de cada ejecución (el de Locust se regeneró cuatro veces después de los datos), con lo que una suma nunca puede delatar un cambio. Hay dos juegos de resultados de Spark distintos ($T(1) = 352,75$ s frente a 235,22 s) sin declarar cuál es el canónico, y `docs/experimentos/resultados/` es una copia. No hay diccionario de datos. En Windows `generar_reporte_correl.py` escribe CRLF y la suma no coincide. `correr_campana.py` valida la cabecera después de recrear el contenedor y de `TRUNCATE`, y borra `verdad_campo.csv` sin respaldo. **Cómo debe quedar:** un árbol canónico único declarado en `resultados/README.md`; manifiestos que cubren todos los datos crudos, derivados y guiones, generados por un comando independiente que nunca se ejecuta al analizar; diccionario con columna, tipo, unidad, dominio y guion productor; `newline=""` en toda escritura; precondiciones validadas antes de cualquier efecto; `job reproduce` que regenera tablas y figuras y termina con `git diff --exit-code`. **Cómo lo verifico:** `grep -n SHA256SUMS experimentos/correr_campana.py resultados/locust/analizar_resultados.py` vacío; `sha256sum -c` en cada carpeta; el número de archivos de datos es igual al de líneas de los manifiestos.

§17 Evaluación ISO/IEC 25010 — tres características sin dato crudo

Qué encontré: eficiencia se reportó con 30 usuarios durante 60 s ($n = 3$), no con 50; fiabilidad exige una hora continua y **nunca se midió más de 119 s seguidos** (unos 15 minutos en total), y la causa atribuida a la licencia de CockroachDB no tiene registro versionado; seguridad es “30/30” con $n = 1$, sin artefacto crudo, sobre dos endpoints y sin inventario; mantenibilidad cita 542/668 y 2.529/3.518, cifras que no están en ningún archivo (JaCoCo dice 555/684 y 2.582/3.582), y la complejidad de `lizard` no está versionada; compatibilidad dice “9/9 Playwright” sin informe, WebKit no es Safari y el móvil no se ejecutó en API 26. En el manuscrito, los “19–34 %” de error son totales sin aplicar los descartes declarados y sin DE ni IC. El contraste con los umbrales (No / No / Parcial) sí es honesto. **Cómo debe quedar:** las cinco características con métrica objetiva, umbral, $n \geq 8$ réplicas válidas donde haya variabilidad, media, DE e IC95 % calculados por un guion versionado desde artefactos crudos versionados: fiabilidad con carga nominal de 60 minutos continuos; seguridad recorriendo todos los endpoints de OpenAPI sin testigo y con testigo inválido, más un informe ZAP; mantenibilidad con JaCoCo de los cuatro servicios y `lizard`; compatibilidad con informes de Playwright y ejecución en emuladores API 26 y 34; tabla del documento generada desde `iso25010.csv`. **Cómo lo verifico:** `python resultados/iso25010/calculador.py && git diff --exit-code`; duración del CSV de fiabilidad ≥ 3.600 s; `grep -rn "542/668\|2529/3518" docs` vacío.

§18 Manuscrito — cifras que no salen de la evidencia

Qué encontré: compila sin indefinidas, pero `pruebas_cicd.tex` declara “90/90 backend, 78/78 web, 5/5 móvil” (hay 121 pruebas Java y 75 web, y “5/5 móvil” contradice la amenaza que admite que las instrumentadas no se ejecutaron) y la discusión habla de “174 pruebas”; los porcentajes de Locust ignoran los descartes; mantenibilidad usa cifras inexistentes (§17). Observabilidad no tiene ninguna figura; la arquitectura sólo muestra C4 de niveles 1 y 2; la tabla ISO no tiene DE ni IC; el resumen repite “Abstract / Resumen”. `ieeetr` no imprime DOI ni ISBN (el PDF no contiene ninguno), hay diez referencias sin identificador y el ISBN de `kim2021devops` no existe. No hay versión congelada. La tabla de aporte no declara los commits generados por guion. **Cómo debe quedar:** conteos de pruebas y tablas ISO generados desde los informes y resultados canónicos; figuras del tablero bajo carga, de una traza y del C4 de nivel 3; `IEEEtran.bst` (0 `biblatex-ieee`) con DOI/ISBN visibles, 12 o más verificados e ISBN corregidos; URL canónica en portada; PDF congelado en `docs/entregas/E4/` con su suma y adjunto al Release de la etiqueta de cierre, compilado *después* de la última regeneración de resultados. **Cómo lo verifico:**

```
cd docs/latex && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "Rerun\|undefined" main.log # 0
pdftotext main.pdf - | grep -ci "doi" # >= 12
pdftotext main.pdf - | grep -n "90/90\|78/78\|174 pruebas\|542/668" # vacío
```

§19 Amenazas a la validez y reflexión ética

Qué encontré: hay tres listas de amenazas (E3, ISO y proyecto); en la de E3 sólo una tiene mitigación aplicada, en la ISO ninguna, y en la del proyecto las dos externas no tienen mitigación, la interna 1 (“un solo desarrollador por capa”) contradice la tabla de aporte y la 3 se apoya en un archivo que no existe en ningún commit. Ninguna trata las amenazas reales de las campañas E4: réplicas deterministas, pseudorreplicación, carga distinta de la exigida y protocolo redactado después de medir. La reflexión ética tiene unas 221 palabras, cita principios genéricos sin numerar y afirma que el sistema “procesa datos personales reales”, en contradicción con los datos sintéticos declarados. **Cómo debe quedar:** una única sección con ≥ 3 internas y ≥ 2 externas que incluya las de las campañas E4, cada una con su mitigación aplicada y el artefacto que la respalda; reflexión de al menos media página con principios ACM numerados (1.2, 1.3, 1.6, 2.5) aplicados a la foto y ubicación del técnico, a los datos de abonados, al mínimo privilegio en JWT y a la honestidad sobre autoría y uso de IA.

§20 Trabajo en equipo y autoría — el historial no permite atribuir el aporte

Qué encontré: el commit `10703f1` añadió `auto_commits_timed.py`, que cambia `user.name` y `user.email` por los de los cuatro integrantes y crea 16 commits con pausas de 15–20 minutos; esos commits coinciden con la serie del 26/07, cuatro de ellos a nombre de Álvarez Párraga. El 21/08 se añadió `ejecutar_commits.py` a `.gitignore`. En la reescritura del 09–10/09, commits de Carpio Mendoza reaparecen con otro autor y *patch-id* idéntico (`df72093` → `286f9b3`, a nombre de Álvarez Párraga; `0f22773` → `1f6f016`). Las actas son reconstruidas el 04/09 desde un chat, registran asistencia completa por política y terminan el 01/09. Con su cuenta propia (`DeJere`), Álvarez Párraga tiene hasta el corte seis commits menores: 124 líneas de documentación, 35 de `.gitattributes`, una recompilación del PDF y un commit vacío. **Cómo debe quedar:** `.mailmap` que unifique las identidades de cada persona; nota de transparencia en actas y manuscrito sobre los commits generados por guion y los reatribuidos, con la tabla de aporte recalculada según ese método; actas levantadas el día de cada reunión del periodo del suspenso; y, para el estudiante que rinde el examen, un bloque sustantivo de los ítems de esta guía (sugeridos: §8, §9 y §12) hecho desde su propia cuenta, en PR revisados por otro integrante, que pueda explicar y defender. Sólo ese trabajo acredita su aporte. **Cómo lo verifico:**

```
git shortlog -sne --no-merges 964fa6e..v4.0.0 # con .mailmap
git log --author=DeJere --numstat 964fa6e..v4.0.0 -- apps/mobile services
```

```
grep -n "auto_commits_timed\|reatribu" docs/actas_reunion.md docs/latex/secciones/*.tex
```

y en la API, PR de DeJere aprobados por otro usuario y PR de otros revisados por DeJere.

4. Orden de ejecución

§3 protección de main y §20 .mailmap y flujo por PR (antes de tocar nada) → §4 dominio sin Spring → §11 migraciones → §8 cierre con evidencia y §5 rutas y SLA → §6 vista de SLA → §10 contratos y OpenAPI → §9 y §7 → §12 y §13 compuertas → §15 protocolo, código congelado y campañas (capturando §14 durante la carga) → §16 paquete de datos → §17 ISO/IEC 25010 → §18 y §19 con las cifras definitivas → §1 y §2 → CHANGELOG, manifiestos finales y, lo último, la etiqueta anotada con su Release.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP HEAD
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main && test -f CHANGELOG.md
grep -rln "import org.springframework" services/*/src/main/java/ec/edu/uteq/soporte/*/domain
grep -n "Multipart\|latitude" apps/mobile/app/src/main/java/ec/edu/uteq/soporte/mobile/data/remote/TicketApi.kt
ls pacts/
for d in $(git ls-files | grep SHA256SUMS.txt | xargs -n1 dirname); do (cd $d && sha256sum -c --quiet
↪ SHA256SUMS.txt) || echo FALLA $d; done
cd docs/latex && latexmk -pdf main.tex && pdftotext main.pdf - | grep -ci doi
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.